

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weightage: 10%

QUESTIONS: 50

Total Marks:50

Mode: In Class

Annexure A

MCQ

Code of Conduct:

I _K.Madhulika(192524190) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Madhulika

Q. No	Conceptual Understanding (30)	Accuracy of Answer (25)	Application of Knowledge (20)	Completeness of Response (15)	Clarity & Presentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 30	/ 25	/20	/ 15	/10	

Annexure A

MCQ

1. What is the main purpose of code optimization?

- A) Increase program size
- B) Improve program performance
- C) Increase compilation time
- D) Add comments

2. Which of the following is a principal source of optimization?

- A) Common subexpressions
- B) Comments
- C) Variable names
- D) Blank lines

3. Optimization that does not change the meaning of a program is called:

- A) Syntax optimization
- B) Semantics-preserving optimization
- C) Lexical optimization
- D) Parsing optimization

4. Peephole optimization is performed on:

- A) Entire source program
- B) Small sequences of target instructions
- C) Only input statements
- D) Symbol table

5. Which is an example of peephole optimization?

- A) Removing redundant instructions
- B) Parsing expressions
- C) Building a syntax tree
- D) Lexical analysis

6. The optimization that replaces expensive operations with cheaper ones is called:

- A) Constant folding
- B) Strength reduction
- C) Dead code elimination
- D) Loop unrolling

7. Which optimization evaluates constant expressions at compile time?

- A) Constant folding
- B) Register allocation
- C) Loop interchange
- D) Copy propagation

8. Example of constant folding is:

- A) $x = y + z$
- B) $x = 5 * 4 \rightarrow x = 20$
- C) $x = y$
- D) $x = x + 1$

9. Dead code elimination removes:

- A) Frequently executed code
- B) Code whose result is never used

- C) All loops
- D) All variables

10. A basic block is a sequence of statements with:

- A) Multiple entry points
- B) One entry and one exit
- C) No entry point
- D) Multiple exit points only

11. The first statement of a basic block is called a:

- A) Leader
- B) Token
- C) Operator
- D) Handle

12. Which of the following can be a leader of a basic block?

- A) First statement of the program
- B) Statement immediately following a conditional jump
- C) Target of a jump
- D) All of the above

13. A flow graph represents:

- A) Lexical structure
- B) Control flow among basic blocks
- C) Data types
- D) Source code comments

14. In a flow graph, nodes generally represent:

- A) Tokens
- B) Basic blocks
- C) Operators
- D) Registers

15. Edges in a flow graph represent:

- A) Data types
- B) Control flow
- C) Variables
- D) Instructions only

16. Global data-flow analysis is used to determine:

- A) Program layout
- B) Information about values and variables across basic blocks
- C) Keywords
- D) Syntax errors

17. Which is a data-flow analysis problem?

- A) Reaching definitions
- B) Lexical analysis
- C) Parsing
- D) Token recognition

18. Reaching definitions determine:

- A) Which definitions may reach a point in the program
- B) Number of registers
- C) Number of tokens
- D) Size of source code

19. Live-variable analysis determines whether a variable:

- A) Is syntactically correct
- B) May be used in the future

C) Is declared globally

D) Is a constant

20. A variable is live at a point if:

A) It has been declared

B) Its current value may be used later

C) It is stored in memory

D) It is a constant

21. Next-use information indicates:

A) The next instruction in the program

B) The next use of a variable

C) The next basic block

D) The next token

22. Next-use information is particularly useful for:

A) Register allocation

B) Lexical analysis

C) Syntax analysis

D) Error reporting

23. A DAG for a basic block is used mainly for:

A) Local optimization

B) Lexical analysis

C) Parsing

D) Memory allocation

24. DAG stands for:

A) Directed Acyclic Graph

B) Data Analysis Graph

C) Directed Algorithm Graph

D) Dynamic Allocation Graph

25. A DAG contains:

A) Cycles always

B) No directed cycles

C) Only one node

D) Only constants

26. DAG representation can identify:

A) Common subexpressions

B) Keywords

C) Syntax errors

D) Comments

27. Consider $a = b + c$; $d = b + c$. The expression $b + c$ is:

A) Dead code

B) Common subexpression

C) Loop invariant

D) Constant

28. Common subexpression elimination avoids:

A) Recomputing the same expression

B) Declaring variables

C) Using registers

D) Creating basic blocks

29. Which optimization moves loop-invariant computations outside a loop?

A) Constant folding

B) Loop-invariant code motion

- C) Copy propagation
- D) Peephole optimization

30. Copy propagation replaces:

- A) A variable by another variable having the same value
- B) Constants by variables
- C) Operators by operands
- D) Loops by conditions

31. Which of the following is a machine-independent optimization?

- A) Constant folding
- B) Register allocation
- C) Instruction selection
- D) Target-specific instruction scheduling

32. Machine-dependent optimization depends on:

- A) Target machine architecture
- B) Source language keywords
- C) Comments
- D) Identifiers

33. The code generator converts:

- A) Machine code to source code
- B) Intermediate representation to target code
- C) Source code to tokens
- D) Tokens to grammar

34. Which is an important issue in code generator design?

- A) Instruction selection
- B) Register allocation
- C) Evaluation order
- D) All of the above

35. The target machine refers to:

- A) The source language
- B) The machine for which target code is generated
- C) The compiler itself
- D) The symbol table

36. Register allocation decides:

- A) Which variables should reside in registers
- B) Which keywords are valid
- C) Which tokens are operators
- D) Which statements are comments

37. Register assignment determines:

- A) Which register is assigned to a value
- B) Which variable is declared
- C) Which statement is a leader
- D) Which operator is used

38. Runtime storage management deals with:

- A) Allocation and organization of memory during execution
- B) Lexical analysis
- C) Parsing
- D) Code comments

39. Which area is commonly associated with runtime storage?

- A) Stack
- B) Heap

- C) Static area
- D) All of the above

40. The stack is commonly used for:

- A) Procedure activation records
- B) Permanent source code
- C) Keywords
- D) Tokens

41. An activation record is associated with:

- A) A procedure or function call
- B) A token
- C) A grammar rule
- D) A comment

42. Which of the following may be stored in an activation record?

- A) Return address
- B) Parameters
- C) Local variables
- D) All of the above

43. A simple code generator commonly uses:

- A) Register and address descriptors
- B) Only a parser
- C) Only a lexical analyzer
- D) Only a symbol table

44. An address descriptor keeps track of:

- A) Locations where the current value of a name can be found
- B) Keywords
- C) Grammar rules
- D) Basic block numbers only

45. A register descriptor keeps track of:

- A) Values currently stored in registers
- B) Variables declared in source code
- C) Operators
- D) Tokens

46. Which instruction is generally used to transfer a value from memory to a register?

- A) LOAD
- B) STORE
- C) JUMP
- D) COMPARE

47. Which instruction transfers a value from a register to memory?

- A) LOAD
- B) STORE
- C) ADD
- D) BRANCH

48. Peephole optimization is called local because it examines:

- A) A small window of consecutive instructions
- B) The entire program
- C) Only the symbol table
- D) All source files

49. Which optimization reduces the number of instructions by eliminating unnecessary computations?

- A) Redundant code elimination

B) Lexical analysis

C) Parsing

D) Type checking

50. The main objective of both code generation and optimization is to produce:

A) Larger source programs

B) Efficient target code

C) More tokens

D) More syntax trees

RUBRICS FOR CALCULATION

Criteria	Weight age (%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

26/8/26]

COS AT2 - MCQ

CSA1419 - Compiler Design

Name: K. Madhulika

Reg no: 192524190

1. [B] Improve program performance
2. [A] common subexpressions
3. [A] syntax optimization
4. [B] small sequences of target instructions
5. [A] Removing redundant instructions
6. [B] Strength reduction
7. [A] Constant folding
8. [B] $X = 5 \times 4 \rightarrow X = 20$
9. [B] code where result is never used
10. [A] Multiple entry points
11. [A] Leader
12. [D] All of the above
13. [B] control flow among Basic blocks
14. [B] Basic blocks
15. [B] Control flow
16. [B] Information about values and variables across basic blocks
17. [A] Reaching definitions

18. [A] Which definitions may reach a point in the program
19. [A] Is syntactically correct
20. [C] It is stored in memory
21. [A] The next instruction in the program
22. [C] syntax analysis
23. [D] Memory allocation
24. [A] Directed Acyclic graph
25. [A] cycles always
26. [A] common subexpressions
27. [B] common subexpression
28. [A] Recomputing the same expression
29. [B] loop invariant code motion
30. [A] A variable by another variable having the same value.
31. [A] constant folding
32. [A] Target machine architecture
33. [B] Intermediate representation to target code
34. [D] All of the above
35. [B]
36. [A]
37. [A]

- 38. [A]
- 39. [D] All of the above
- 40. [D] Tokens
- 41. [A] A procedure or function call
- 42. [D] All of the above
- 43. [A] Register and address descriptors
- 44. [A] Locations where the current value of a name is stored
- 45. [A] values currently stored in registers
- 46. [A] LOAD
- 47. [A] LOAD
- 48. [A] A small window of consecutive expressions
- 49. [A] Redundant code elimination
- 50. [B] Efficient target code